

Tutoriel PyQt5

Guide complet : widgets, signaux, layouts & exemples

Ce tutoriel couvre les fondamentaux de PyQt5 : installation, widgets, le mécanisme signaux & slots, les layouts et un exemple d'application complète.

1. Introduction

PyQt5 est un ensemble de bindings Python pour le framework Qt5. Il permet de créer des interfaces graphiques (GUI) riches et multiplateformes avec Python, en combinant la simplicité de Python et la puissance de Qt.

Points forts

Multiplateforme	Windows, macOS et Linux avec le même code source.
Riche en widgets	Boutons, tableaux, graphiques, boîtes de dialogue, etc.
Signaux & Slots	Système événementiel puissant et très intuitif.
Layouts flexibles	Mise en page automatique qui s'adapte au redimensionnement.

Structure de base

```
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow

class MaFenetre(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Mon app PyQt5")
        self.setGeometry(100, 100, 600, 400) # x, y, largeur, hauteur

app = QApplication(sys.argv) # 1. Créer l'application
fenetre = MaFenetre()       # 2. Créer la fenêtre
fenetre.show()               # 3. L'afficher
sys.exit(app.exec_())        # 4. Lancer la boucle événementielle
```

2. Installation

```
# Installation via pip
pip install PyQt5

# Avec les outils de design (Qt Designer)
pip install PyQt5 PyQt5-tools

# Vérifier l'installation
python -c "import PyQt5; print('Installation OK')"
```

Imports typiques

```
from PyQt5.QtWidgets import (
    QApplication, QMainWindow, QWidget,
    QPushButton, QLabel, QLineEdit,
    QVBoxLayout, QHBoxLayout, QGridLayout
)
from PyQt5.QtCore import Qt, pyqtSignal
from PyQt5.QtGui import QFont, QIcon, QColor
```

Astuce : Utilisez un environnement virtuel (`python -m venv venv`) avant d'installer PyQt5 pour éviter les conflits de versions.

3. Widgets

Les widgets sont les éléments visuels de votre interface. Voici les plus couramment utilisés :

Widget	Description	Signal principal
<code>QPushButton</code>	Bouton cliquable	<code>clicked</code>
<code>QLabel</code>	Affiche texte ou image	—
<code>QLineEdit</code>	Champ de saisie une ligne	<code>textChanged</code> , <code>returnPressed</code>
<code>QTextEdit</code>	Zone de texte multi-lignes	<code>textChanged</code>
<code>QComboBox</code>	Liste déroulante	<code>currentIndexChanged</code>
<code>QCheckBox</code>	Case à cocher	<code>stateChanged</code>
<code>QSlider</code>	Curseur numérique	<code>valueChanged</code>
<code>QSpinBox</code>	Saisie numérique avec flèches	<code>valueChanged</code>

Exemples d'utilisation

```
from PyQt5.QtWidgets import *
from PyQt5.QtCore import Qt

# Label centré
label = QLabel("Bonjour !")
label.setAlignment(Qt.AlignCenter)

# Bouton avec taille fixe
btn = QPushButton("Cliquer ici")
btn.setFixedSize(120, 36)

# Champ de saisie avec placeholder
champ = QLineEdit()
champ.setPlaceholderText("Entrez votre nom...")

# Liste déroulante
combo = QComboBox()
combo.addItems(["Python", "JavaScript", "Rust"])

# Zone de texte multi-lignes
texte = QTextEdit()
texte.setText("Contenu modifiable...")
```

4. Signaux & Slots

Le mécanisme Signaux & Slots est le cœur de PyQt5. Un **signal** est émis par un widget lors d'un événement ; un **slot** est la fonction appelée en réponse.

Connexions de base

```
# Connexion simple : signal -> slot
btn.clicked.connect(ma_fonction)

# Avec une lambda pour passer des arguments
btn.clicked.connect(lambda: print("Cliqué !"))

# Signal avec valeur (ex : QSlider)
slider.valueChanged.connect(lambda v: label.setText(f"Valeur : {v}"))
```

Récupérer et stocker les saisies utilisateur

C'est l'un des usages les plus courants. Deux approches selon le besoin :

```
# Approche 1 – en temps réel (textChanged)
self.champ = QLineEdit()
self.valeur = ""

self.champ.textChanged.connect(self.stocker)

def stocker(self, texte):
    self.valeur = texte          # mis à jour à chaque frappe

# Approche 2 – à la validation (bouton ou Entrée)
btn.clicked.connect(self.recuperer)
self.champ.returnPressed.connect(self.recuperer)

def recuperer(self):
    self.valeur = self.champ.text() # lu au moment voulu
    print(f"Valeur : {self.valeur}")

# QTextEdit – textChanged ne passe pas le texte directement
self.zone = QTextEdit()
self.zone.textChanged.connect(self.lire_zone)

def lire_zone(self):
    self.contenu = self.zone.toPlainText()
```

Signaux courants par widget

Widget	Signal	Description
--------	--------	-------------

QPushButton	clicked	Bouton cliqué
QLineEdit	textChanged(str)	Texte modifié (en temps réel)
QLineEdit	returnPressed	Touche Entrée pressée
QCheckBox	stateChanged(int)	État coché / décoché
QSlider	valueChanged(int)	Valeur du curseur changée
QComboBox	currentIndexChanged	Sélection changée
QTextEdit	textChanged	Contenu modifié (appeler toPlainText())

Signaux personnalisés

```

from PyQt5.QtCore import QObject, pyqtSignal

class MonObjet(QObject):
    # Déclaration d'un signal personnalisé
    donnee_prete = pyqtSignal(str)

    def traiter(self, valeur):
        resultat = valeur.upper()
        self.donnee_prete.emit(resultat)    # Émettre le signal

# Connexion
obj = MonObjet()
obj.donnee_prete.connect(lambda s: print(s))
obj.traiter('bonjour')    # affiche : BONJOUR

```

5. Layouts

Les layouts organisent automatiquement les widgets dans la fenêtre. Ils s'adaptent au redimensionnement sans avoir besoin de coordonnées fixes.

QVBoxLayout	Empile les widgets verticalement. Idéal pour formulaires et panneaux.
HBoxLayout	Aligne les widgets horizontalement. Parfait pour les barres d'outils.
GridLayout	Organise en lignes et colonnes. Pour des formulaires complexes.
FormLayout	Paies label / champ. Spécialisé pour les formulaires de saisie.

```
# Layout vertical
vbox = QVBoxLayout()
vbox.addWidget(label)
vbox.addWidget(champ)
vbox.addWidget(btn)

# Layout horizontal
hbox = QHBoxLayout()
hbox.addWidget(QPushButton("OK"))
hbox.addWidget(QPushButton("Annuler"))

# Imbriquer des layouts
principal = QVBoxLayout()
principal.addLayout(hbox)
principal.addStretch() # espace flexible

# Grille
grid = GridLayout()
grid.addWidget(label, 0, 0) # ligne 0, col 0
grid.addWidget(champ, 0, 1) # ligne 0, col 1
grid.addWidget(btn, 1, 0, 1, 2) # occupe 2 colonnes
```

Astuce : Appliquez le layout sur un QWidget central : `widget.setLayout(layout)`, puis `self.setCentralWidget(widget)` dans votre QMainWindow.

6. Exemple complet

Un convertisseur de température Celsius → Fahrenheit qui illustre tous les concepts vus : QMainWindow, widgets, signaux, layouts.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtCore import Qt

class Convertisseur(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Convertisseur de température")
        self.setFixedSize(350, 200)

        # Widgets
        self.champ = QLineEdit()
        self.champ.setPlaceholderText("Entrez une température en °C")

        self.resultat = QLabel("Résultat : -")
        self.resultat.setAlignment(Qt.AlignCenter)

        btn = QPushButton("Convertir")
        btn.clicked.connect(self.convertir)
        self.champ.returnPressed.connect(self.convertir)

        # Layout
        layout = QVBoxLayout()
        layout.setContentsMargins(20, 20, 20, 20)
        layout.setSpacing(12)
        layout.addWidget(QLabel("Celsius → Fahrenheit"))
        layout.addWidget(self.champ)
        layout.addWidget(btn)
        layout.addWidget(self.resultat)

        container = QWidget()
        container.setLayout(layout)
        self.setCentralWidget(container)

    def convertir(self):
        try:
            celsius = float(self.champ.text())
            fahrenheit = celsius * 9/5 + 32
            self.resultat.setText(f"{celsius}°C = {fahrenheit:.1f}°F")
        except ValueError:
            self.resultat.setText("Entrez un nombre valide")

app = QApplication(sys.argv)
fenetre = Convertisseur()
fenetre.show()
sys.exit(app.exec_())
```

Pour aller plus loin

Menus & barre d'état	QMenuBar, QStatusBar, QToolBar
Boîtes de dialogue	QMessageBox, QFileDialog, QInputDialog
Modèles de données	QTableWidget, QListWidget, QTreeWidget
Multithreading	QThread + signaux pour les tâches longues
Qt Designer	Créer des interfaces visuellement (.ui → Python)
Stylisation	Qt StyleSheets (QSS) pour personnaliser l'apparence
